



ET0732 - MLAI MINI PROJECT

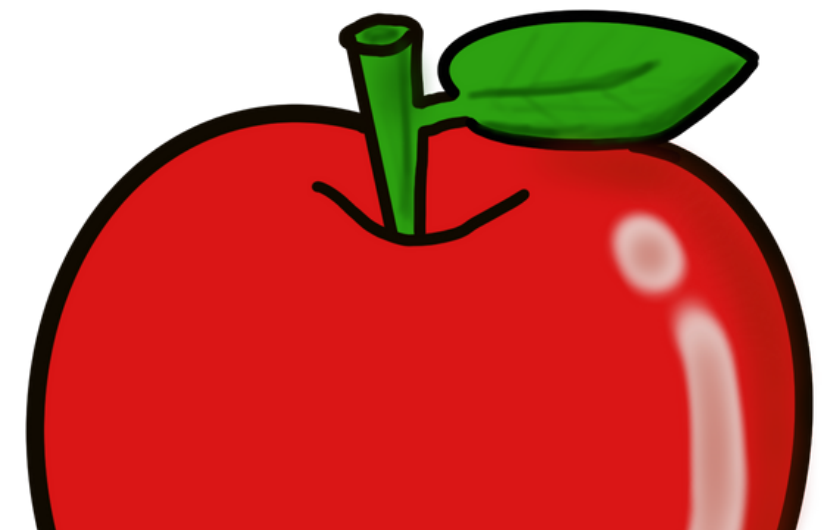
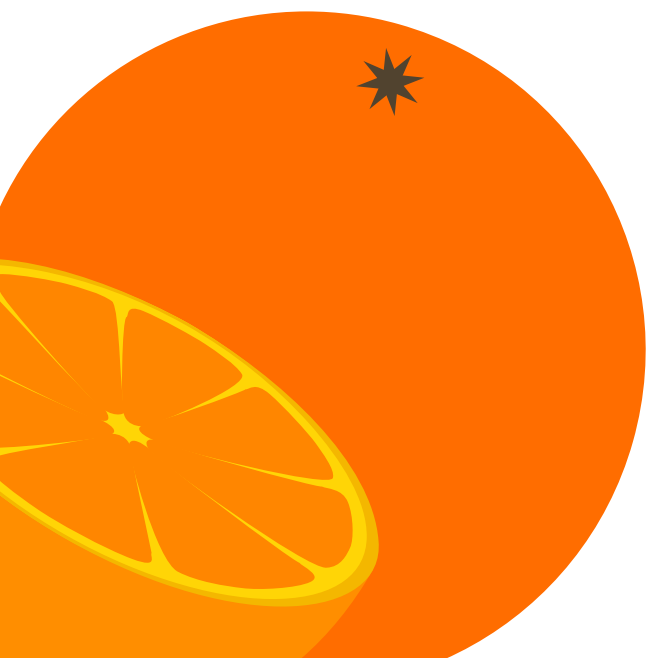
ORANGE & RED APPLE

Wai Yan Aung 2112422

Stanley Lin 2120795

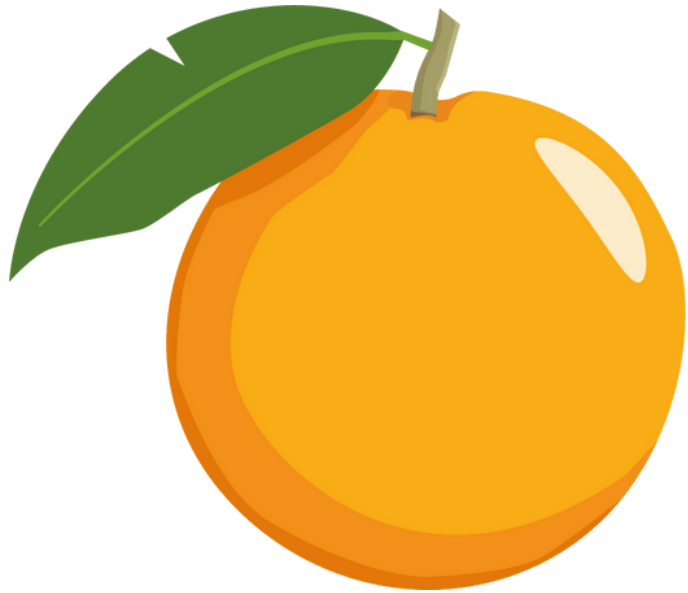
Class: DCPE/FT/3A/03

ET0732: MACHINE LEARNING&AI



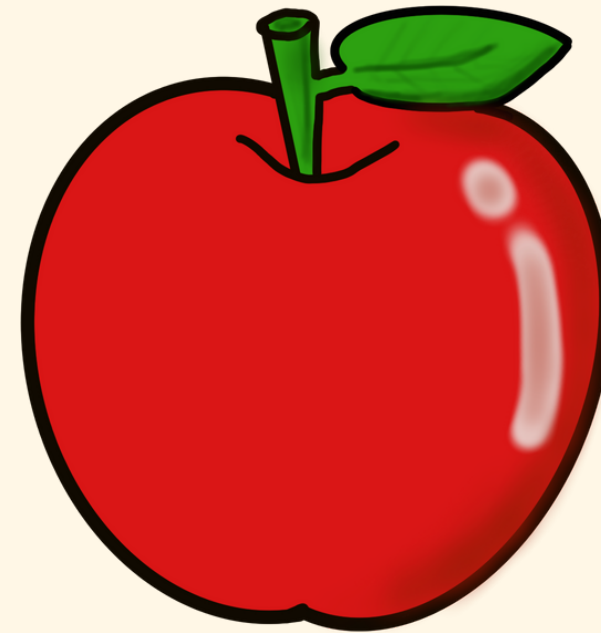
CATEGORIES

ORANGE



-REAL-LIFE PHOTOS
OF ORANGES TAKEN
BY STANLEY

RED APPLE



-REAL-LIFE PHOTOS
OF APPLES TAKEN BY
WAI YAN AUNG

UNKNOWN



-PHOTOS OF SURROUNDINGS
AND OTHER UNRELATED
OBJECTS TAKEN BY BOTH
WAI YAN AUNG AND STANLEY

DATA COLLECTION

We included real-life photos of Apples, Oranges and other objects.

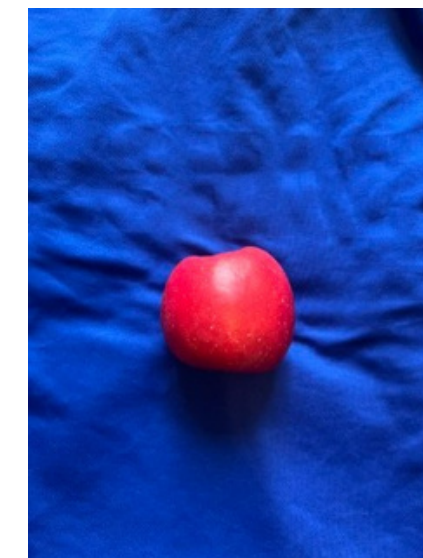
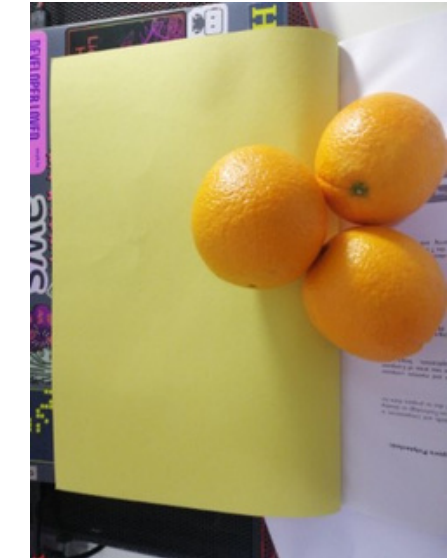
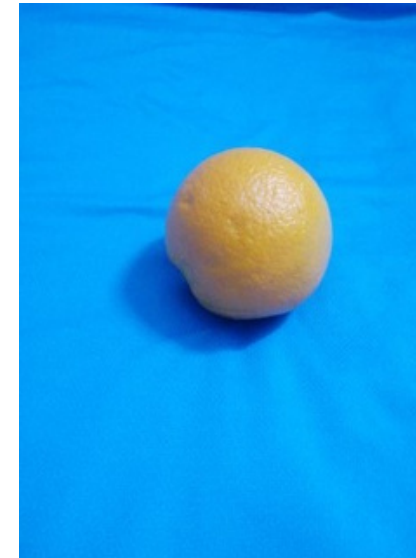
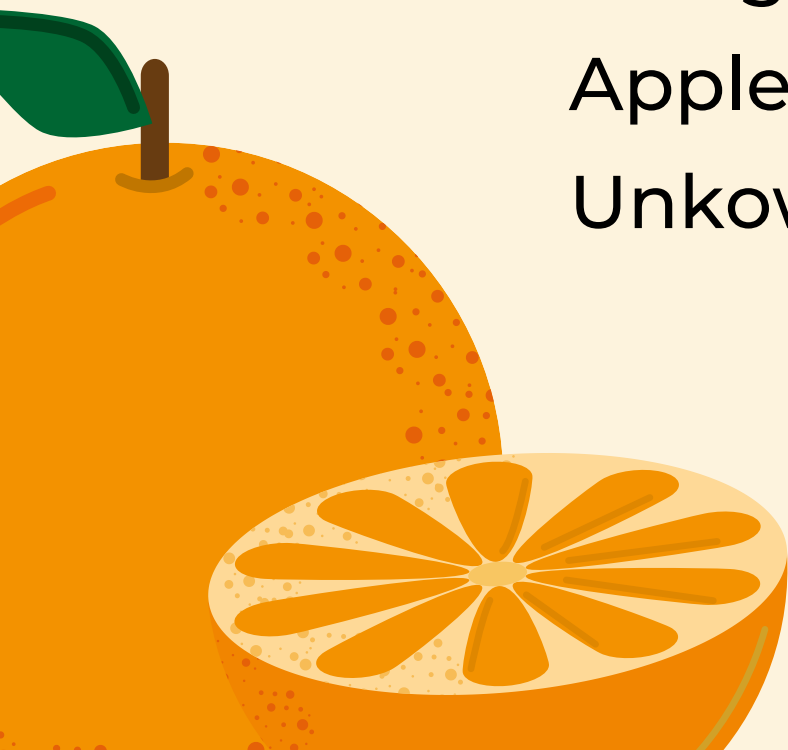
We burst shot a series of pictures of the fruits from various angles, lighting and background colour.

Total photos taken:

Orange: 1577

Apple: 1606

Unkown: 1377



DATASET PREPARATION

- Images taken were resized using **PowerToys** to 400x400 to scale down the size of images to train model faster.

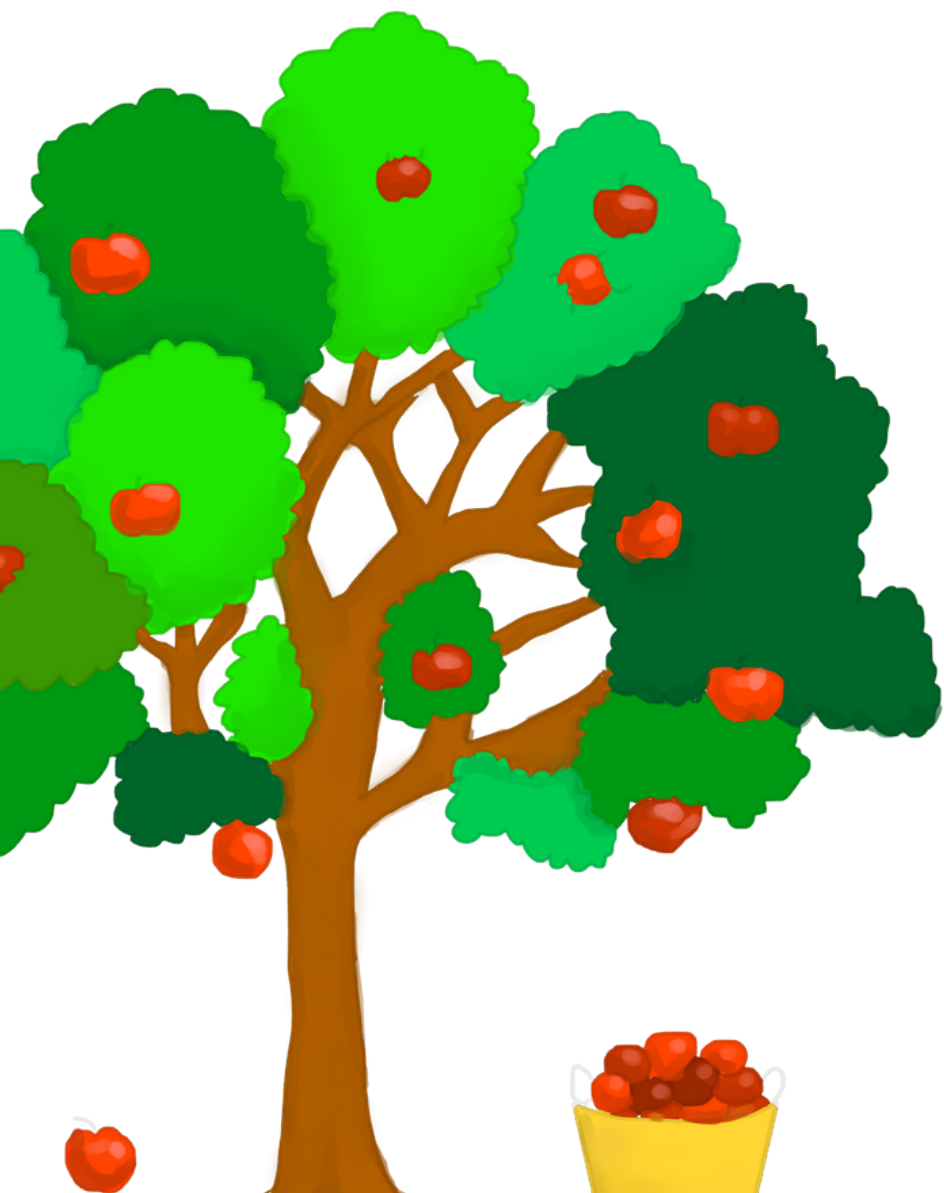


IMAGE RESIZER

A screenshot of the Windows Image Resizer application window. The window has a title bar with the text "Image Resizer" and a close button. Below the title bar, there is a section "Select a size" with a dropdown menu showing "Custom". Below this, there are four input fields: "Resize type" with a dropdown showing "Fit", "Width" with a numeric input of "400", "Height" with a numeric input of "400", and "Unit" with a dropdown showing "Pixel". Below these fields, there are four checkboxes: "Make pictures smaller but not larger" (unchecked), "Ignore the orientation of pictures" (checked), "Resize the original pictures (don't create copies)" (checked), and "Remove metadata that doesn't affect rendering" (unchecked). At the bottom right, there are two buttons: "Resize" and "Cancel".

INSIDE DATASETV1

A screenshot of the DATASETV1 directory structure. It shows a list of folders: "Apple", "Orange", and "Unknown". Each folder is represented by a yellow folder icon and its name.

© 2006 The Authors
Journal compilation © 2006 Blackwell Publishing Ltd

-
- A close-up illustration of a red apple with a bite taken out of it, showing the white flesh and a small seed.

IMG_20230621_1 95835_BURST020 (Custom).jpg	IMG_20230621_1 95835_BURST021 (Custom).jpg	IMG_20230621_1 95835_BURST022 (Custom).jpg	IMG_20230621_1 95835_BURST023 (Custom).jpg	IMG_20230621_1 95835_BURST024 (Custom).jpg	IMG_20230621_1 95835_BURST025 (Custom).jpg	IMG_20230621_1 95835_BURST026 (Custom).jpg	IMG_20230621_1 95835_BURST027 (Custom).jpg	IMG_20230621_1 95835_BURST028 (Custom).jpg	IMG_20230621_2 00022_BURST001 _COVER (Custom).jpg	IMG_20230621_2 00022_BURST002 (Custom).jpg
IMG_20230621_2 00022_BURST003 (Custom).jpg	IMG_20230621_2 00022_BURST004 (Custom).jpg	IMG_20230621_2 00022_BURST005 (Custom).jpg	IMG_20230621_2 00022_BURST006 (Custom).jpg	IMG_20230621_2 00022_BURST007 (Custom).jpg	IMG_20230621_2 00022_BURST008 (Custom).jpg	IMG_20230621_2 00022_BURST009 (Custom).jpg	IMG_20230621_2 00022_BURST010 (Custom).jpg	IMG_20230621_2 00022_BURST011 (Custom).jpg	IMG_20230621_2 00022_BURST012 (Custom).jpg	IMG_20230621_2 00022_BURST013 (Custom).jpg
IMG_20230621_2 00022_BURST014 (Custom).jpg	IMG_20230621_2 00022_BURST015 (Custom).jpg	IMG_20230621_2 00028_BURST001 _COVER (Custom).jpg	IMG_20230621_2 00028_BURST002 (Custom).jpg	IMG_20230621_2 00028_BURST003 (Custom).jpg	IMG_20230621_2 00028_BURST004 (Custom).jpg	IMG_20230621_2 00028_BURST005 (Custom).jpg	IMG_20230621_2 00028_BURST006 (Custom).jpg	IMG_20230621_2 00028_BURST007 (Custom).jpg	IMG_20230621_2 00028_BURST008 (Custom).jpg	IMG_20230621_2 00028_BURST009 (Custom).jpg
IMG_20230621_2 00028_BURST010 (Custom).jpg	IMG_20230621_2 00028_BURST011 (Custom).jpg	IMG_20230621_2 00028_BURST012 (Custom).jpg	IMG_20230621_2 00028_BURST013 (Custom).jpg	IMG_20230621_2 00028_BURST014 (Custom).jpg	IMG_20230621_2 00028_BURST015 (Custom).jpg	IMG_20230621_2 00028_BURST016 (Custom).jpg	IMG_20230621_2 00210 (Custom).jpg	IMG_20230621_2 00220_BURST001 _COVER (Custom).jpg	IMG_20230621_2 00220_BURST002 (Custom).jpg	IMG_20230621_2 00220_BURST003 (Custom).jpg

0004.JPG	IMG_0012.JPG	IMG_0013.JPG	IMG_0035.JPG	IMG_0041.JPG	IMG_0053.JPG	IMG_0054.JPG	IMG_0069.JPG	IMG_0070.JPG	IMG_0071.JPG
00124.JPG	IMG_0137.JPG	IMG_0144.JPG	IMG_0147.JPG	IMG_0148.JPG	IMG_0167.JPG	IMG_0168.JPG	IMG_0169.JPG	IMG_0172.JPG	IMG_0173.JPG
00213.JPG	IMG_0228.JPG	IMG_0229.JPG	IMG_0232.JPG	IMG_0243.JPG	IMG_0269.JPG	IMG_0270.JPG	IMG_0286.JPG	IMG_0287.JPG	IMG_0288.JPG
00331.JPG	IMG_0350.JPG	IMG_0351.JPG	IMG_0373 - Copy.JPG	IMG_0373.JPG	IMG_0374 - Copy.JPG	IMG_0374.JPG	IMG_0384 - Copy.JPG	IMG_0384.JPG	IMG_0385.JPG

IMG_0854 - Copy (Custom).JPG	IMG_0863 - Copy - Copy (2) (Custom).JPG	IMG_0864 - Copy (2) - Copy (Custom).JPG	IMG_0865 - Copy (Custom).JPG	IMG_0869 - Copy - Copy - Copy (Custom).JPG	IMG_0869 - Copy (3) (Custom).JPG	IMG_0869 - Copy (Custom).JPG	IMG_0878 - Copy - Copy (Custom).JPG	IMG_0883 - Copy - Copy (2) (Custom).JPG	IMG_0883 - Copy (2) (Custom).JPG	IMG_0883 - Copy - Copy (Custom).JPG
IMG_0896 - Copy (2) (Custom).JPG	IMG_0897 (Custom).JPG	IMG_0898 - Copy (2) - Copy (Custom).JPG	IMG_0900 - Copy (2) - Copy (2) (Custom).JPG	IMG_0900 - Copy (2) - Copy (Custom).JPG	IMG_0906 - Copy - Copy (Custom).JPG	IMG_0910 - Copy - Copy - Copy (Custom).JPG	IMG_0913 (Custom).JPG	IMG_0914 - Copy - Copy - Copy (Custom).JPG	IMG_0914 - Copy (3) (Custom).JPG	IMG_0914 - Copy (Custom).JPG
IMG_0928 - Copy (Custom).JPG	IMG_0943 (Custom).JPG	IMG_0944 - Copy (2) - Copy (Custom).JPG	IMG_0947 - Copy (3) (Custom).JPG	IMG_0956 - Copy - Copy - Copy (Custom).JPG	IMG_0956 - Copy - Copy (Custom).JPG	IMG_0960 - Copy (2) - Copy (Custom).JPG	IMG_0961 - Copy (2) - Copy (Custom).JPG	IMG_0962 (Custom).JPG	IMG_0970 - Copy - Copy (Custom).JPG	IMG_1000 - Copy - Copy (Custom).JPG
IMG_1031 (Custom).JPG	IMG_1032 - Copy - Copy (3) - Copy (Custom).JPG	IMG_1046 - Copy - Copy (4) - Copy (Custom).JPG	IMG_1046 - Copy (7) - Copy (Custom).JPG	IMG_1047 - Copy - Copy (3) - Copy (Custom).JPG	IMG_1047 - Copy (2) - Copy (Custom).JPG	IMG_1048 - Copy - Copy - Copy (Custom).JPG	IMG_1056 - Copy (2) (Custom).JPG	IMG_1101 - Copy (Custom).JPG	IMG_1106 - Copy (Custom).JPG	IMG_1106 - Copy (Custom).JPG

LIBRARIES USED

- 1) Tensorflow
- 2) Pillow version 8.4.0
- 3) Numpy
- 6) Matplotlib
- 7) cv2 (Open Computer vision)



DATA SPILTTING

1) Prepared dataset called DatasetV1 consisting of (Apple + Orange + Unkown) and was split into Training, Test, and Validation sets using **Sklearn train_test_split** function.

- 5% of original dataset used as Test
- 5% of original dataset used as Validation

2)To copy the content of source files to the appropriate destination file which is our DatasetV2, we utilized the **shutil.copy()** method.

```
import os
import shutil

#-----
#DATA SPLITTING

#DatasetV1 is our 400x400 images, it contains 3 directories Orange, Apple, Unknown.
orange_src = 'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV1/Orange/'
apple_src= 'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV1/Apple/'
unknown_src='C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV1/Unknown/'

orange_data=os.listdir(orange_src)
apple_data=os.listdir(apple_src)
unknown_data=os.listdir(unknown_src)

from sklearn.model_selection import train_test_split

#DatasetV1's 3 directories Orange, Apple, Unknown is then distributed randomly into Test, Train, Valid. For example, Test file would
#contain the 3 directories Orange, Apple, Unknown. The same can be said for Train and Valid files, these are all contained in DatasetV2.

orange_train,orange_test = train_test_split(orange_data,test_size=0.05,random_state=1) #Trasnferring 5% of the data,and randomizing
orange_train,orange_validate=train_test_split(orange_train,test_size=0.05,random_state=1)

apple_train,apple_test=train_test_split(apple_data,test_size=0.05,random_state=1)
apple_train,apple_validate=train_test_split(apple_train,test_size=0.05,random_state=1)

unknown_train,unknown_test=train_test_split(unknown_data,test_size=0.05,random_state=1)
unknown_train,unknown_validate=train_test_split(unknown_train,test_size=0.05,random_state=1)

#After the random split, the images are sorted into DatasetV2
for image in orange_train:
    shutil.copy(orange_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Train/Orange/')
for image in orange_test:
    shutil.copy(orange_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Test/Orange/')
for image in orange_validate:
    shutil.copy(orange_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Valid/Orange/')

for image in apple_train:
    shutil.copy(apple_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Train/Apple/')
for image in apple_test:
    shutil.copy(apple_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Test/Apple/')
for image in apple_validate:
    shutil.copy(apple_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Valid/Apple/')

for image in unknown_train:
    shutil.copy(unknown_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Train/Unknown/')
for image in unknown_test:
    shutil.copy(unknown_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Test/Unknown/')
for image in unknown_validate:
    shutil.copy(unknown_src+image,'C:/Python/2120795_St StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Valid/Unknown/')
```

DATA SPILTTING

- **DatasetV1**'s 3 directories
Orange, Apple, and Unknown
is then distributed randomly
into Test, Train, and Valid into
DatasetV2. For example, the
Test directory would contain
the 3 directories Orange,
Apple, and Unknown.

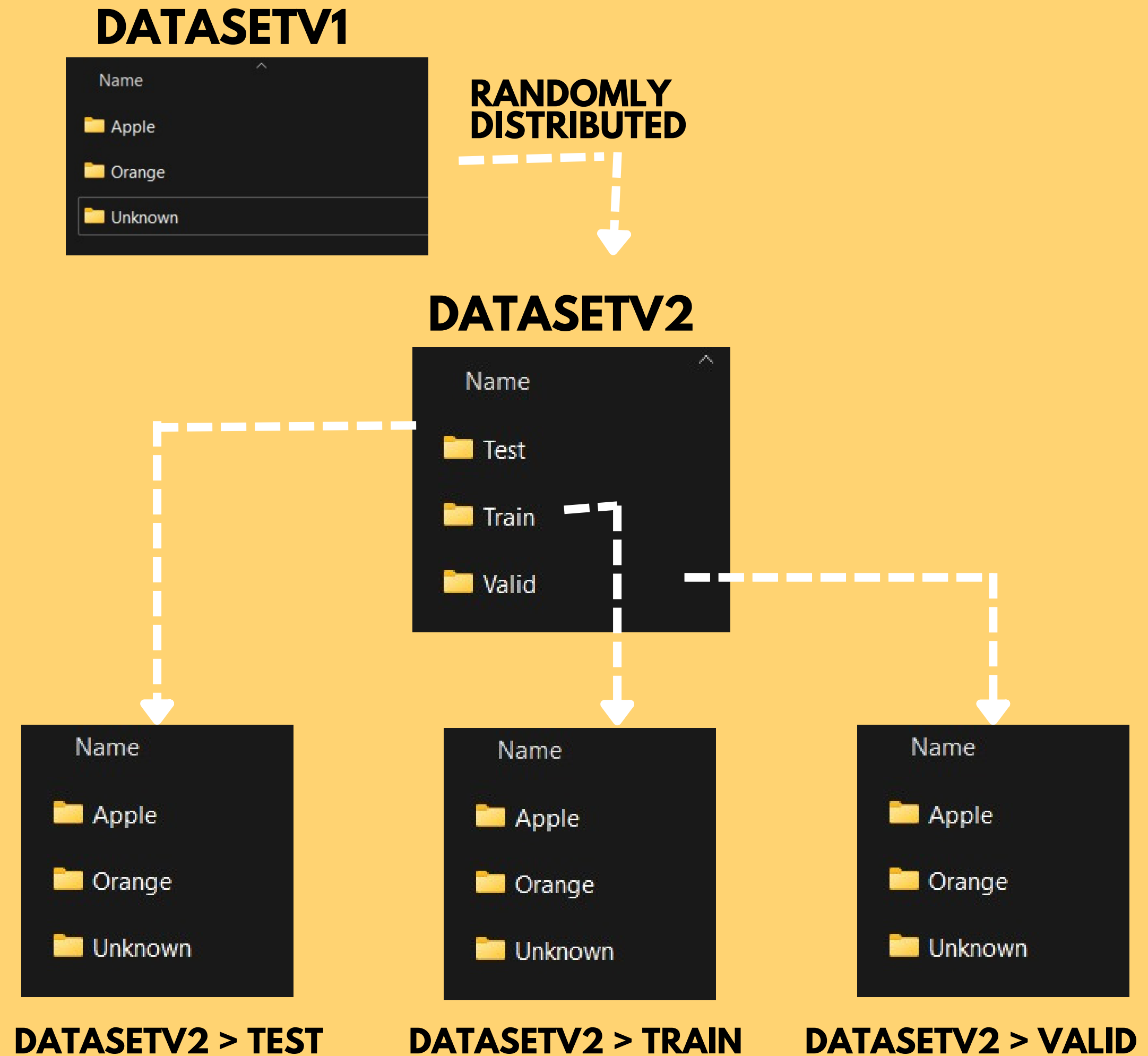


IMAGE PRE-PROCESSING

- In order to scale down the image for pre-processing and augmenting images, we used these specifications to train more quickly.

-**rescale**=1./255, scales the pixel values of the images to be in the range of 0 to 1.

-**rotation_range**=20 randomly rotates the images by a maximum of 20 degrees.

-**width_shift_range**=0.2 randomly shifts the images horizontally by a fraction of 0.2 of the total width.

-**height_shift_range**=0.2 randomly shifts the images vertically by a fraction of 0.2 of the total height.

-**horizontal_flip**=True randomly flips the images horizontally.

-**fill_mode**='nearest' fills in any newly created pixels during image transformations with the nearest existing pixels.

```
import tensorflow as tf
```

```
#Rescale 1./255 changes the pixel values from 0 -255 to 0 -1.
```

```
#This will help with the speed of the training and increase fairness
```

```
train_datagen=tf.keras.preprocessing.image.ImageDataGenerator(  
    rescale=1./255,  
    rotation_range = 20,  
    #zoom_range = 0.2,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    #validation_split = 0.15  
    horizontal_flip=True,  
    fill_mode='nearest'  
)
```

```
test_datagen = tf.keras.preprocessing.image.ImageDataGenerator(  
    rescale=1./255  
)
```

```
valid_datagen = tf.keras.preprocessing.image.ImageDataGenerator(  
    rescale=1./255  
)
```

IMAGE SHARPENING

- To sharpen our photos inside **DatasetV2**, we mapped our images into a higher dimensional space using the **kernel** approach to apply a linear classifier.
- All 3 Folders(Train, Test, Valid) inside DatasetV2 are being sharpened.
- Sharpened images are then generated inside our **DatasetV3** directory using **NumPy** and **kernel function**.

```
from PIL import Image
from PIL import ImageFilter
import os
import cv2
import numpy as np

#IMAGE SHARPENING
#-----
#PATH OF THE FOLDER CONTAINING THE CODE
inPath_Orange='C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV2/Train/Orange/'

#PATH OF THE FOLDER THAT WILL CONTAIN THE MODIFIED IMAGE
outPath_Orange='C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV3/Train/Orangesharp/'

for imagePath in os.listdir(inPath_Orange):
    #imagePath CONTAINING NAME OF THE IMAGE
    inputPath=os.path.join(inPath_Orange,imagePath)

    #inputPath CONTAINNING THE FULL DIRECTORY
    img=cv2.imread(inputPath,cv2.IMREAD_COLOR)

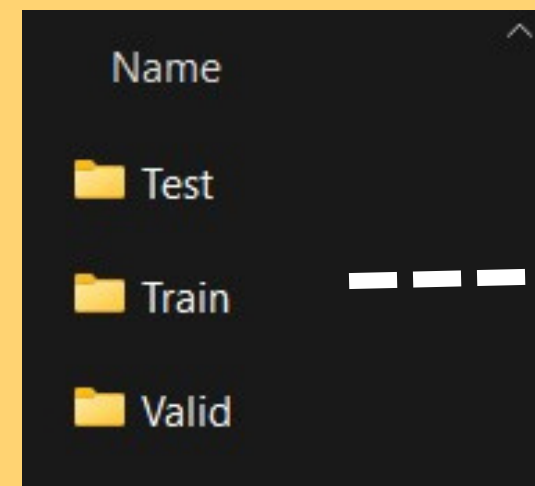
    fullOutPath=os.path.join(outPath_Orange,'sharpened'+imagePath)
    #fullOutPath contains the path of the output
    #image that needs to be generated
    kernel = np.array([[0,-1,0],
                       [-1,5,-1],
                       [0,-1,0]])

    image_sharp=cv2.filter2D(src=img,ddepth=-1,kernel=kernel)

    cv2.imwrite(fullOutPath,image_sharp)

    print(fullOutPath)
```

DATASETV3



DATASETV3 > TRAIN

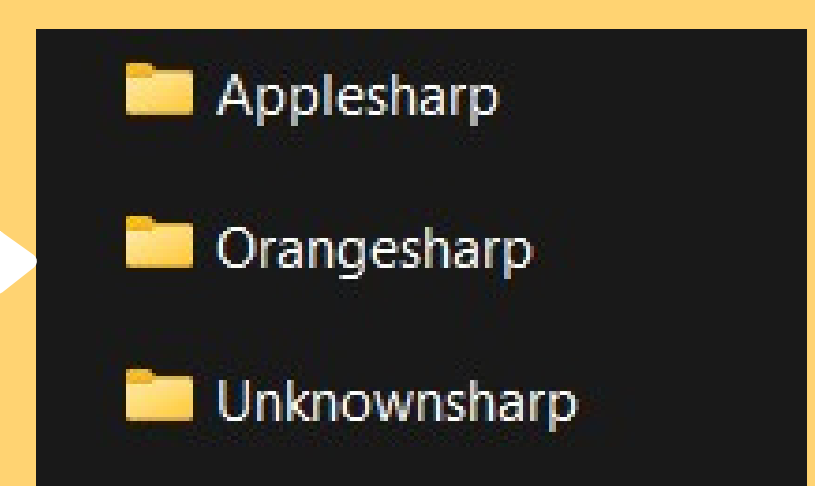
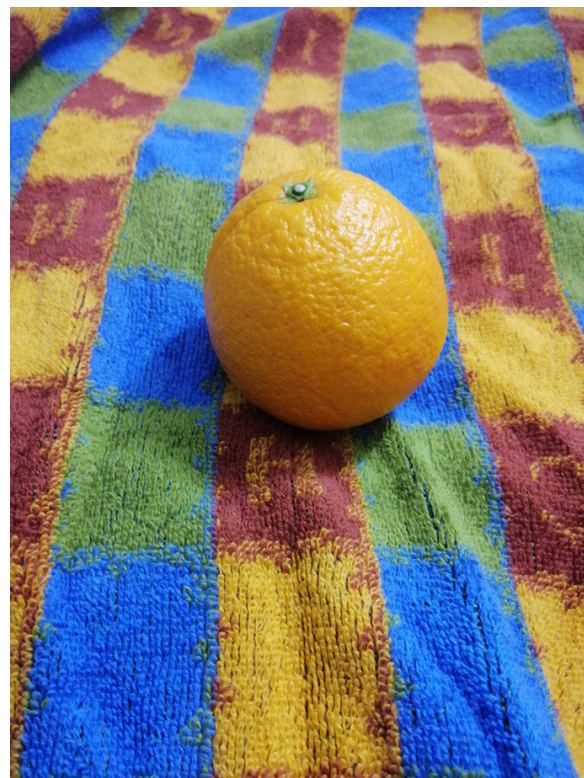


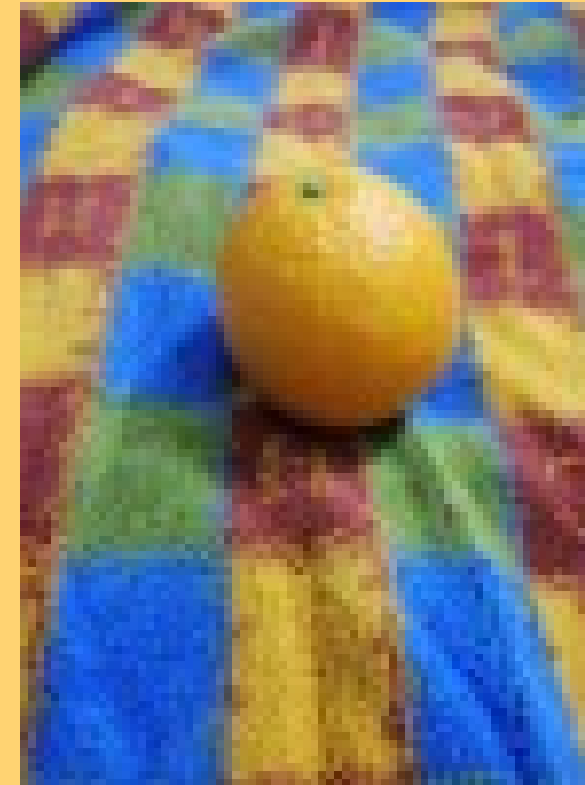
IMAGE SHARPENING

- We perform image sharpening with the intention of ensuring that images are clear and maintain the original quality for training and that images are in small size for shorter training times.

**ORIGINAL IMAGE
SIZE: 3456X4608**



100X100 UNSHARPEN



100X100 SHARPENED



400X400 UNSHARPEN



400X400 SHARPENED



SET VARIABLES AND PREPARE TO TRAIN DATA

- Variables set:

Image size: 224,224

Batch size: 16

Learning rate Optimizer: Adam, $1e^{-5}$

- Directory of **DatasetV3** is loaded using the `flow_from_directory` method.
- `categorical_crossentropy` is used for the loss function
- Images data are converted to tensor which iterates the data .

```
import tensorflow as tf
from tensorflow.keras.layers import Input, Conv2D, Flatten, Dense, MaxPool2D, BatchNormalization
from tensorflow.keras.models import Model, Sequential
from PIL import Image
from PIL import ImageFilter

# Define the size of the input images
IMAGE_SIZE=[224,224]
# Set the batch size for training
BATCH_SIZE=16
# Adam Optimizer with a learning rate of 1e-5
opt=tf.keras.optimizers.Adam(learning_rate=1e-5)

# Directory paths for training, testing, and validation data
train_dir ='C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV3/Train/'
test_dir ='C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV3/Test/'
valid_dir='C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/DatasetV3/Valid/'

# Creating a generator for training data
train_generator = train_datagen.flow_from_directory(train_dir,
                                                    target_size = (IMAGE_SIZE),
                                                    batch_size = BATCH_SIZE,
                                                    class_mode = 'categorical',
                                                    shuffle=True
                                                    # Shuffle the data for each epoch
                                                    )

# Creating a generator for testing data
test_generator = test_datagen.flow_from_directory(test_dir,
                                                  target_size = (IMAGE_SIZE),
                                                  batch_size = BATCH_SIZE,
                                                  class_mode = 'categorical',
                                                  shuffle=False
                                                  # Do not shuffle the data for testing
                                                  )

valid_generator = valid_datagen.flow_from_directory(valid_dir,
                                                    target_size = (IMAGE_SIZE),
                                                    batch_size = BATCH_SIZE,
                                                    class_mode = 'categorical',
                                                    shuffle=False
                                                    # Do not shuffle the data for validation
                                                    )
```

BUILDING THE MODEL

- We defined a function called `scratchmodel` that creates a convolutional neural network (CNN) model using the Keras API with TensorFlow backend.
- This function:

- 1) Takes two arguments `weights` (weight initialization for model) and `include_top` (determines whether to include the top layers of the model).
- 2) Initializes a `sequential model` that allows us to build the model by stacking layers on top of each other sequentially
- 3) `Conv2D` layers and `MaxPool2D` layers consist of various parameters to be trained and learned.
- 4) `Batchnormalization` layer normalizes the activations of the previous layers and helps in stabilizing.
- 5) Returns a constructed model

```
def scratchmodel(weights='imagenet', include_top=False):  
    #Building model in sequential manner  
    model=Sequential()  
  
    #Convulotional Layers  
    model.add(Conv2D(filters=32, input_shape=IMAGE_SIZE+[3], kernel_size=(3,3), padding="same", activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2), strides=(2,2)))  
  
    model.add(Conv2D(filters=64, kernel_size=(3,3), padding="same", activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2), strides=(2,2)))  
  
    model.add(Conv2D(filters=128, kernel_size=(3,3), padding="same", activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2), strides=(2,2)))  
  
    model.add(Conv2D(filters=256, kernel_size=(3,3), padding="same", activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2), strides=(2,2)))  
  
    model.add(tf.keras.layers.BatchNormalization())  
    return model  
  
# Creating the model using the scratchmodel function  
model=scratchmodel()  
  
# Flattening the model output  
x=tf.keras.layers.Flatten()(model.output)  
  
# Adding dropout to reduce overfitting  
x=tf.keras.layers.Dropout(0.6)(x)  
x=tf.keras.layers.Dense(64, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)  
x=tf.keras.layers.Dropout(0.4)(x)  
x=tf.keras.layers.Dense(32, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)  
x=tf.keras.layers.Dropout(0.2)(x)  
x=tf.keras.layers.Dense(3, activation='softmax', kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)  
  
# Creating the final model  
model=Model(inputs=model.input, outputs=x)  
# Compiling the model with the specified optimizer, loss, and metrics  
model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
```

BUILDING THE MODEL

- This part of the code adds additional layers(dropout and dense) to the 'scratchmodel' and compiles the model for training with specified optimizer, loss function and metrics.
- **Flatten layer**: to converts the multi-dimensional tensor into 1-dimensional vector.
- **Dropout layers**: in prevention of overfitting.
- **Softmax activation function**: provides the probability distribution over the classes.
- A new model using functional API of Keras is created.

```
def scratchmodel(weights='imagenet',include_top=False):  
    #Building model in sequential manner  
    model=Sequential()  
  
    #Convulotional layers  
    model.add(Conv2D(filters=32,input_shape=IMAGE_SIZE+[3],kernel_size=(3,3),padding="same",activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))  
  
    model.add(Conv2D(filters=64,kernel_size=(3,3),padding="same",activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))  
  
    model.add(Conv2D(filters=128,kernel_size=(3,3),padding="same",activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))  
  
    model.add(Conv2D(filters=256,kernel_size=(3,3),padding="same",activation="relu"))  
    model.add(MaxPool2D(pool_size=(2,2),strides=(2,2)))  
  
    model.add(tf.keras.layers.BatchNormalization())  
    return model  
  
# Creating the model using the scratchmodel function  
model=scratchmodel()  
  
# Flattening the model output  
x=tf.keras.layers.Flatten()(model.output)  
  
# Adding dropout to reduce overfitting  
x=tf.keras.layers.Dropout(0.6)(x)  
x=tf.keras.layers.Dense(64, activation='relu',kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)  
x=tf.keras.layers.Dropout(0.4)(x)  
x=tf.keras.layers.Dense(32, activation='relu',kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)  
x=tf.keras.layers.Dropout(0.2)(x)  
x=tf.keras.layers.Dense(3,activation='softmax',kernel_regularizer=tf.keras.regularizers.l2(1e-4))(x)  
  
# Creating the final model  
model=Model(inputs=model.input, outputs=x)  
# Compiling the model with the specified optimizer, loss, and metrics  
model.compile(optimizer=opt, loss='categorical_crossentropy', metrics=['accuracy'])
```

BUILDING THE MODEL



- **ModelCheckpoint**: to create checkpoints and save the model on each completed epoch during training. By using the ModelCheckpoint callback, we can save the model during training to track its progress and have the ability to restore the best-performing model later on.

```
#Create checkpoints and save the model on each completed epoch during training
checkpoint=tf.keras.callbacks.ModelCheckpoint(
    "C:/Python/2120795_St StanleyLin_Pineapple_2112422_WaiYanAung_Apple/osu_model_400_e10.hdf5",
    monitor="val_loss",
    verbose=1,
    save_best_only=False,
    save_weights_only=False,
    mode="auto",
    save_freq="epoch",\
    options=None,
)

#Monitors the validation accuracy and stop based on its value
early=tf.keras.callbacks.EarlyStopping(monitor="val_accuracy",
    min_delta=0,
    patience=40,
    verbose=1,
    mode="auto",
)

# Enable logging of device placement for debugging purposes
tf.debugging.set_log_device_placement(True)

try:
    # Specify the GPU device to be used (assuming GPU:0)
    with tf.device('/device:GPU:0'):
        # Training the model
        history = model.fit(
            train_generator,
            steps_per_epoch=len(train_generator),
            epochs=10, #number of training rounds
            validation_data = valid_generator,
            validation_steps=len(valid_generator),
            callbacks=[checkpoint,early]
        )
except RuntimeError as e:
    print(e)
```

BUILDING THE MODEL

- **EarlyStopping**: allows us to stop the training process if the observed validation accuracy does not improve after a certain number of epochs. It helps in the prevention of overfitting and conserves computational resources by terminating training.



```
#Create checkpoints and save the model on each completed epoch during training
checkpoint=tf.keras.callbacks.ModelCheckpoint(
    "C:/Python/2120795_StanleyLin_Pineapple_2112422_WaiYanAung_Apple/osu_model_400_e10.hdf5",
    monitor="val_loss",
    verbose=1,
    save_best_only=False,
    save_weights_only=False,
    mode="auto",
    save_freq="epoch",\
    options=None,
)

#Monitors the validation accuracy and stop based on its value
early=tf.keras.callbacks.EarlyStopping(monitor="val_accuracy",
    min_delta=0,
    patience=40,
    verbose=1,
    mode="auto",
)

# Enable logging of device placement for debugging purposes
tf.debugging.set_log_device_placement(True)

try:
    # Specify the GPU device to be used (assuming GPU:0)
    with tf.device('/device:GPU:0'):
        # Training the model
        history = model.fit(
            train_generator,
            steps_per_epoch=len(train_generator),
            epochs=10, #number of training rounds
            validation_data = valid_generator,
            validation_steps=len(valid_generator),
            callbacks=[checkpoint,early]
        )
except RuntimeError as e:
    print(e)
```

TRAINING THE MODEL

- **model.fit() function**: trains the model by iterating through the training data for the number of epochs. It adjusts the model's parameters depending on the defined optimizer and loss function, analyzes the model's performance on the validation data.
- **history**: The training progress and metrics for each epoch are stored in the history variable, which can be used for analysis and visualization using matplotlib library.

```
#Create checkpoints and save the model on each completed epoch during training
checkpoint=tf.keras.callbacks.ModelCheckpoint(
    "C:/Python/2120795_StanleyLin_Pineapple_2112422_WaiYanAung_Apple/osu_model_400_e10.hdf5",
    monitor="val_loss",
    verbose=1,
    save_best_only=False,
    save_weights_only=False,
    mode="auto",
    save_freq="epoch",\
    options=None,
)

#Monitors the validation accuracy and stop based on its value
early=tf.keras.callbacks.EarlyStopping(monitor="val_accuracy",
    min_delta=0,
    patience=40,
    verbose=1,
    mode="auto",
)

# Enable logging of device placement for debugging purposes
tf.debugging.set_log_device_placement(True)

try:
    # Specify the GPU device to be used (assuming GPU:0)
    with tf.device('/device:GPU:0'):
        # Training the model
        history = model.fit(
            train_generator,
            steps_per_epoch=len(train_generator),
            epochs=10, #number of training rounds
            validation_data = valid_generator,
            validation_steps=len(valid_generator),
            callbacks=[checkpoint,early]
        )
except RuntimeError as e:
    print(e)
```

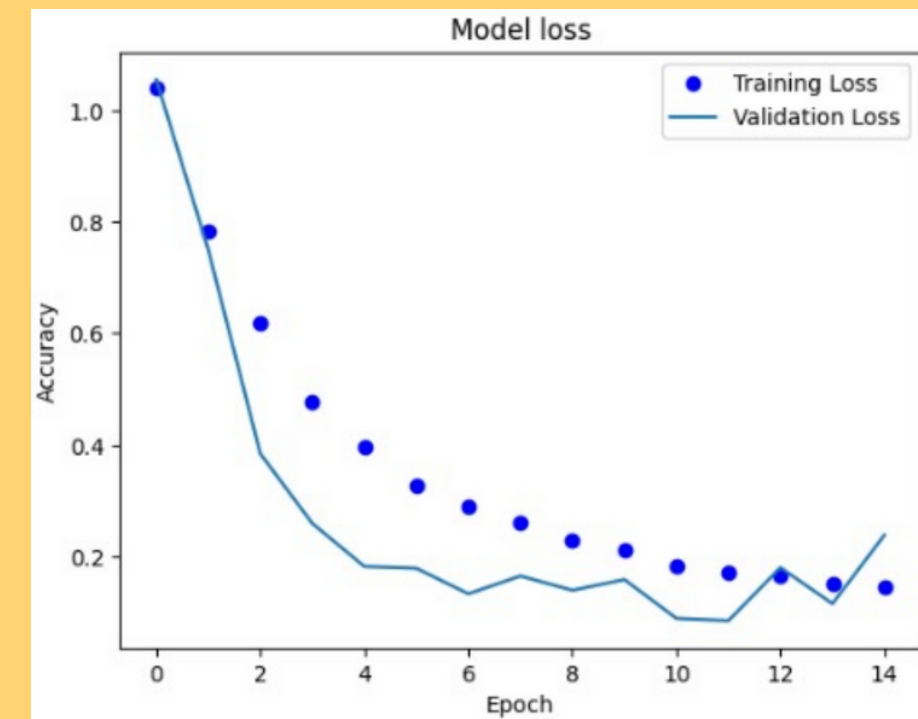
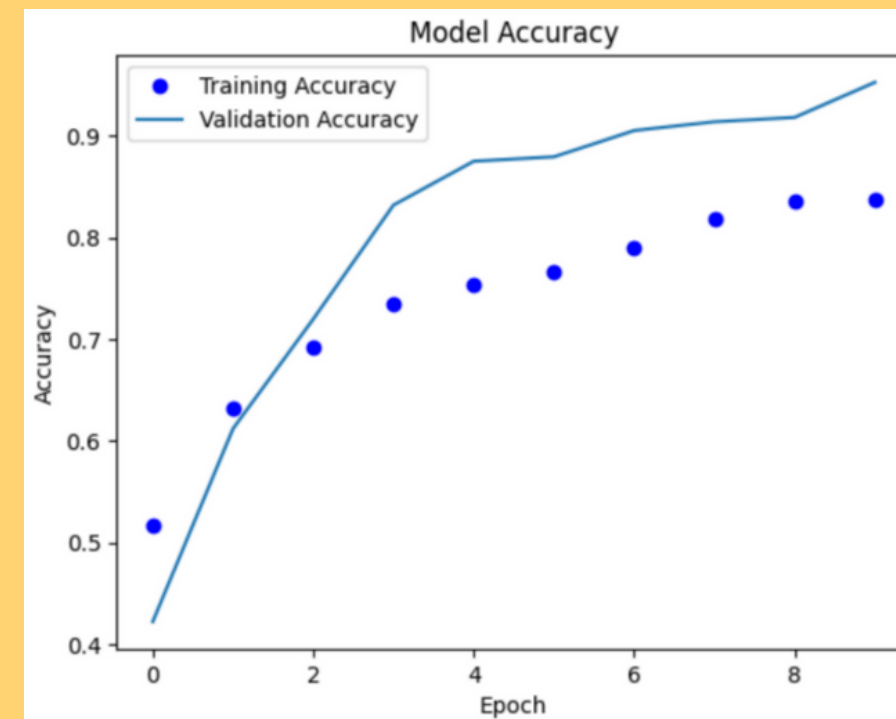
VISUALISATIONS AND PLOTTING OF GRAPH

- Using `matplotlib` library we were able to generate a data visualization on our `history` variable. One for model accuracy and another for model loss. The plots help visualize the performance of the model over the training epochs, allowing you to analyze if the model is overfitting (high training accuracy but low validation accuracy) or underfitting (low training accuracy).

```
import matplotlib.pyplot as plt
#-----
#USED MATPLOTLIB TO DETERMINE IF THE MODEL IS OVERFITTING OR UNDERFITTING
#DETERMINE ACCURACY AND LOSS
plt.plot(history.history["accuracy"], 'bo')
plt.plot(history.history['val_accuracy'])
plt.title("Model Accuracy")
plt.ylabel("Accuracy")
plt.xlabel("Epoch")
plt.legend(["Training Accuracy", "Validation Accuracy"])
plt.figure()

plt.plot(history.history["loss"], 'bo')
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.ylabel("Loss")
plt.xlabel("Epoch")
plt.legend(["Training Loss", "Validation Loss"])

plt.show()
```



STREAMLIT WEB APP IMAGE TESTING

- Using **Spyder** software we created a python file **oau_app.py** which creates a web app that allows users to upload an image file and get predictions for whether the image contains an Apple, an Orange, or is classified as Unknown.



Streamlit

```
import streamlit as st
import tensorflow as tf
import numpy as np
from PIL import Image, ImageOps

def import_and_predict(image_data, model):

    size = (224,224)
    image = ImageOps.fit(image_data, size, Image.ANTIALIAS)
    image = image.convert('RGB')
    image = np.asarray(image)
    image = (image.astype(np.float32) / 255.0)

    img_reshape = image[np.newaxis,...]

    prediction = model.predict(img_reshape)

    return prediction

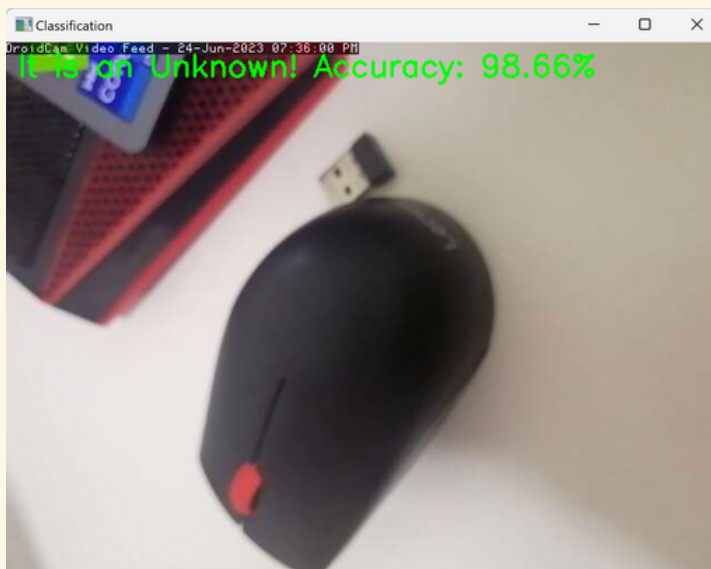
model = tf.keras.models.load_model("C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/oau_model_400_e10.hdf5") #load a trained model
st.write("""
    # Orange-Apple-Unknown Prediction
    """)
st.write("This is a simple image classification web app to predict Orange-Apple-Unknown")
file = st.file_uploader("Please upload an image file", type=["jpg", "png"])

if file is None:
    st.text("You haven't uploaded an image file")
else:
    image = Image.open(file)
    st.image (image, use_column_width=True)
    prediction = import_and_predict(image, model)
    if np.argmax(prediction) == 0:
        st.write("It is an Apple!")
    elif np.argmax (prediction) == 1:
        st.write("It is an Orange!")
    else:
        st.write("It is unknown!")

    st.text("Probability (0: Apple, 1: Orange, 2: Unknown)")
    st.write(prediction)
```

LIVE DETECT

- The laptop webcam has low resolution, unable to detect features of the fruits
- We used Droid Cam with our phone and place our IP Cam Access into cap = cv2.VideoCapture(w)
- Inside Droid Cam Mobile App, there is an IP cam access IP web link, copy that into String variable w.



```
model = tf.keras.models.load_model(
    "C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/oau_model_400_e10.hdf5")
    #Load a trained model

#cap = cv2.VideoCapture(0)
#It will use the default camera of the laptop (E.g. In-built Webcam)

w = 'http://192.16x.xx.xxx:xxx7/video'
cap = cv2.VideoCapture(w)
#It will use DriodCam

if (cap.isOpened()): #Check if Camera/WebCam is available
    print("Camera OK")
else:
    cap.open() #Turn on Camera
while (True):
    ret, original = cap.read()

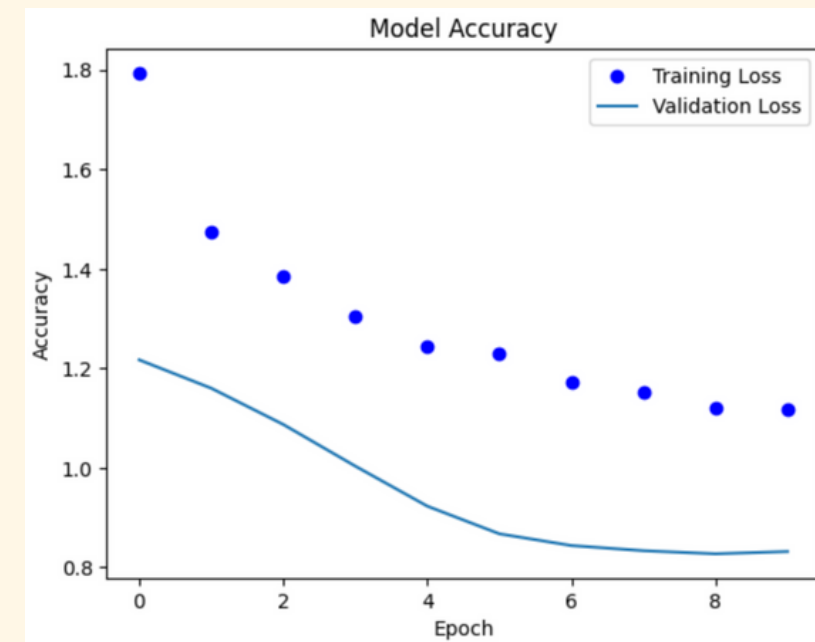
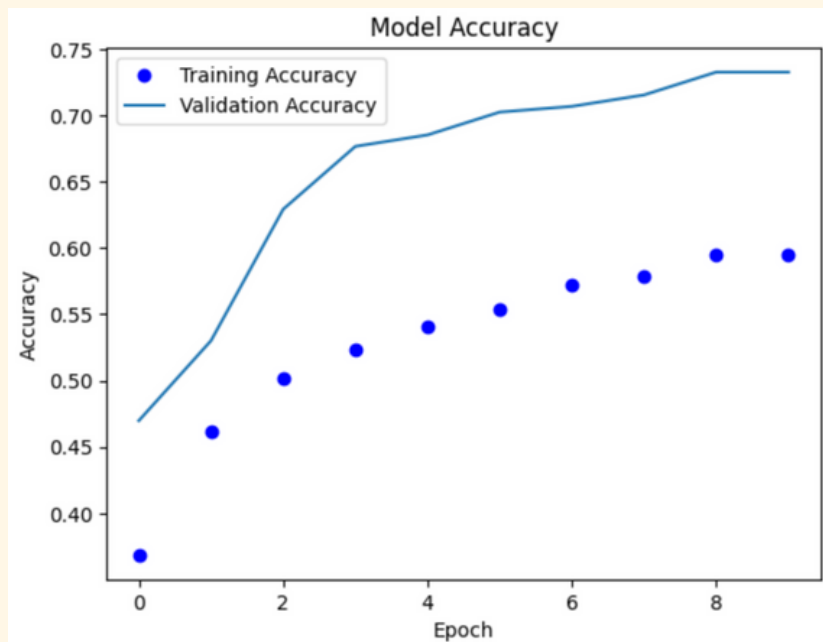
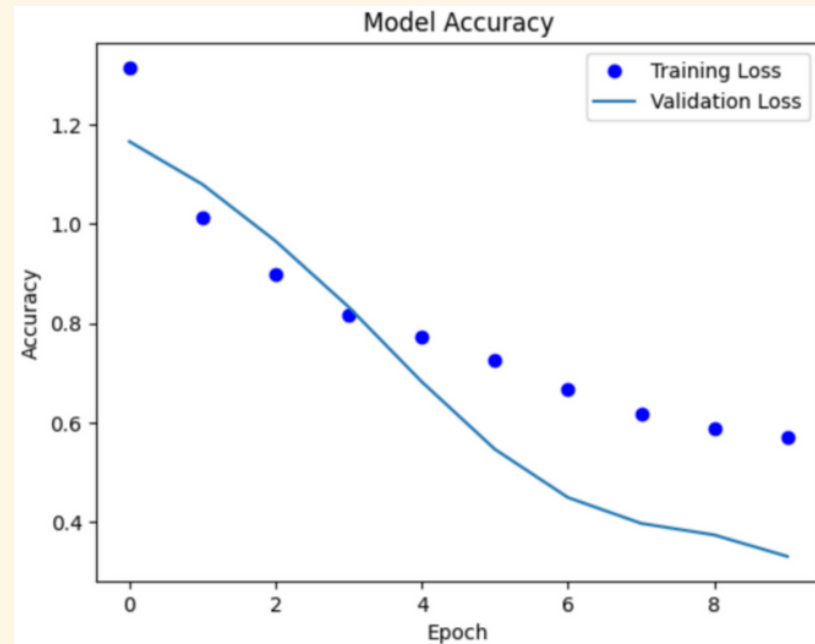
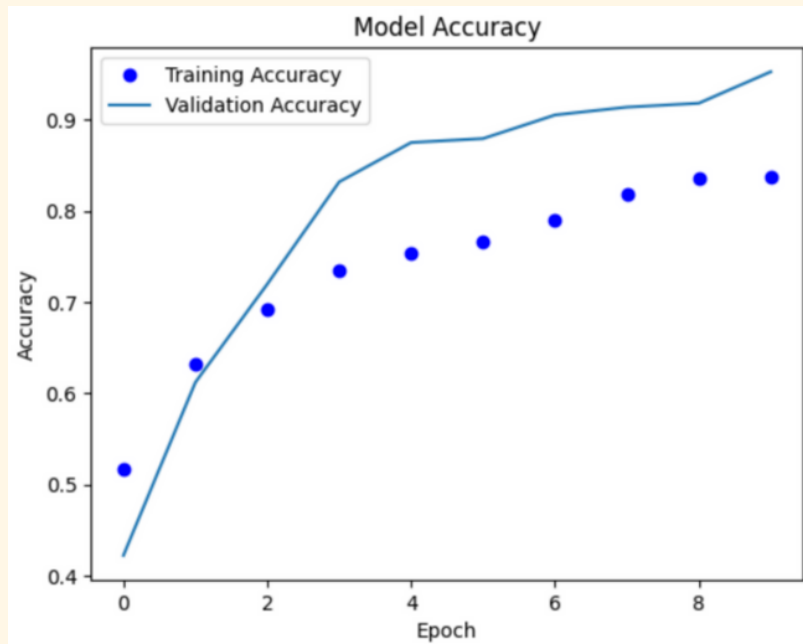
    frame = cv2.resize(original, (224, 224))
    cv2.imwrite(filename='img.jpg', img=original)
    image = Image.open('img.jpg')

    # Display the predictions
    # print("ImageNet ID: {}, Label: {}".format(inID, Label))
    prediction = import_and_predict(image, model)
    #print (prediction)

    if (np.argmax(prediction) != 0) & (np.argmax (prediction) != 1):
        #Check whether it is not Orange and not Apple
        predict="It is an Unknown!"
    elif np.argmax(prediction) == 1: #Check whether it is an Orange
        predict="It is an Orange!"
    elif np.argmax(prediction) == 0: #Check whether it is a Kiwi
        predict="It is an Apple!"
    cv2.putText(original, predict, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.9, (0, 255, 0), 2)
    cv2.imshow("Classification", original)

    if (cv2.waitKey(1) & 0xFF== ord('q')):
        break;
```

PREVIOUS MODELS



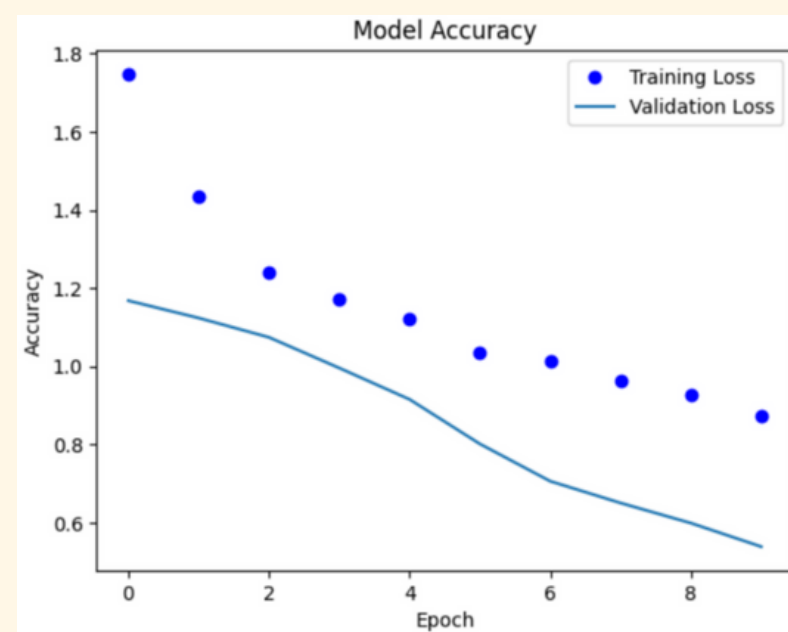
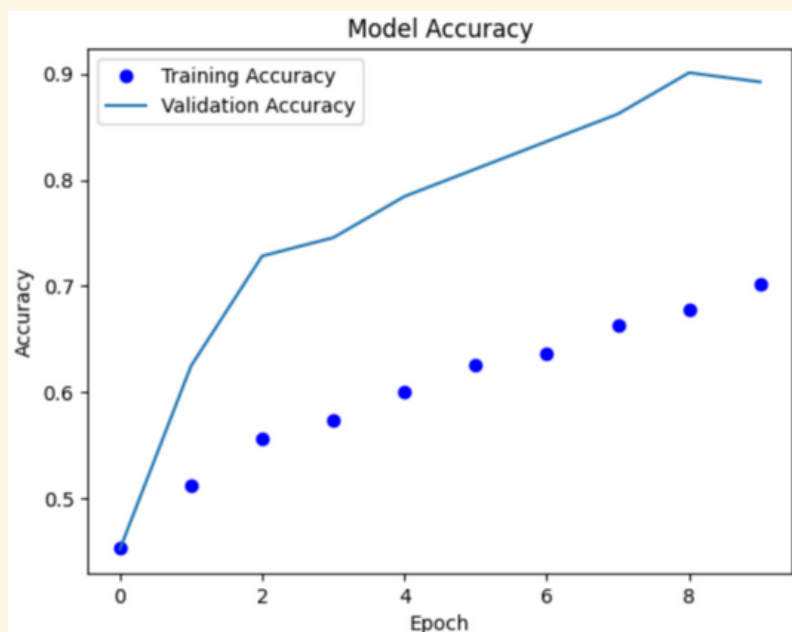
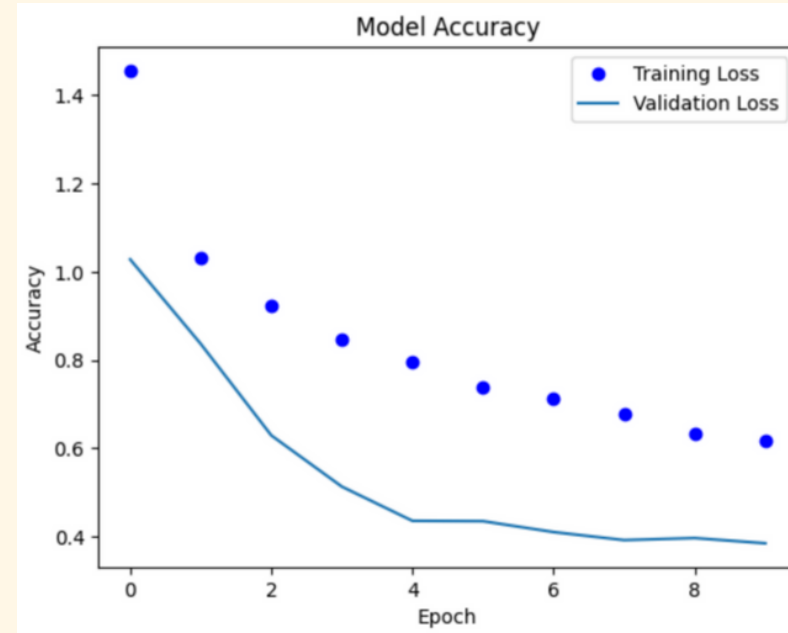
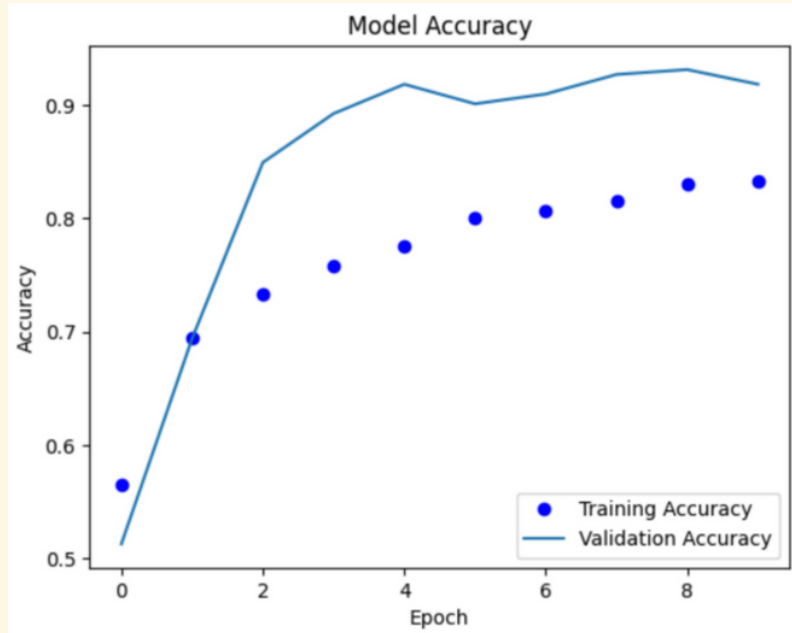
Model #5

- Adam Opt Learning rate: $1e-5$
- Conv2D: 3 Layers with activation "Relu" and max-pooling (32,64,128 filters)
- Dropout: 5 layers (0.5, 0.5, 0.3, 0.3, 0.3)
- Dense: 3 layers (512,256,128)
- Steps per Epoch: `len(train_generator)`
- Epoch:10
- Batch size: 50

Model #6

- Adam Opt Learning rate: $1e-6$
- Conv2D: 3 Layers with activation "Relu" and max-pooling (32,64,128 filters)
- Dropout: 5 layers (0.5, 0.5, 0.3, 0.3, 0.3)
- Dense: 3 layers (512,256,128)
- Steps per Epoch: `len(train_generator)`
- Epoch:10
- Batch size: 50

PREVIOUS MODELS



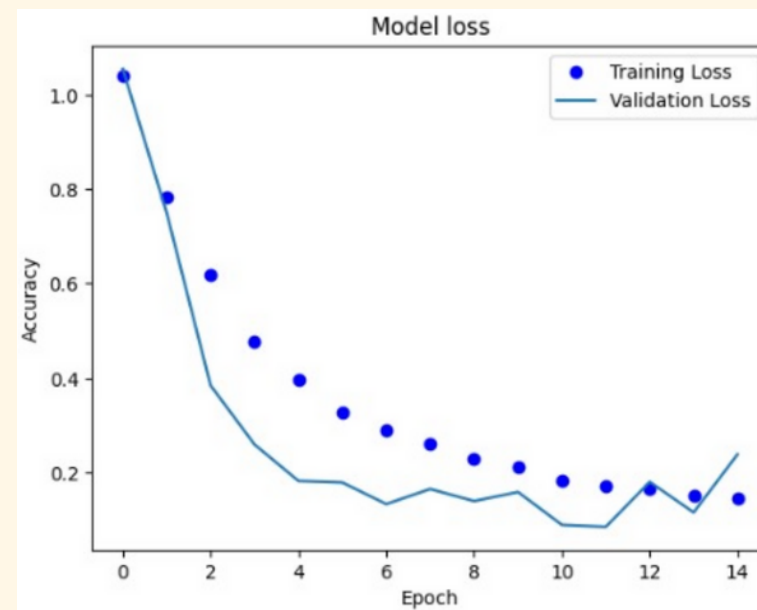
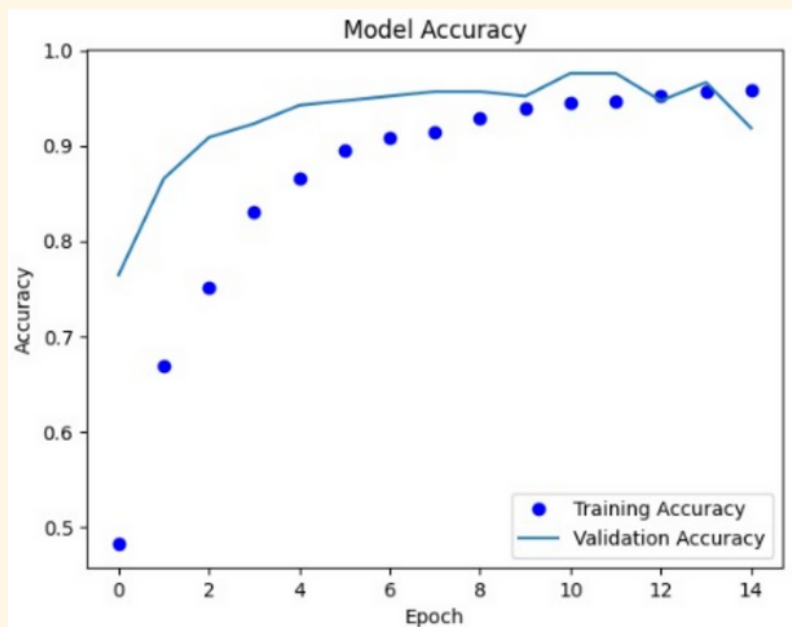
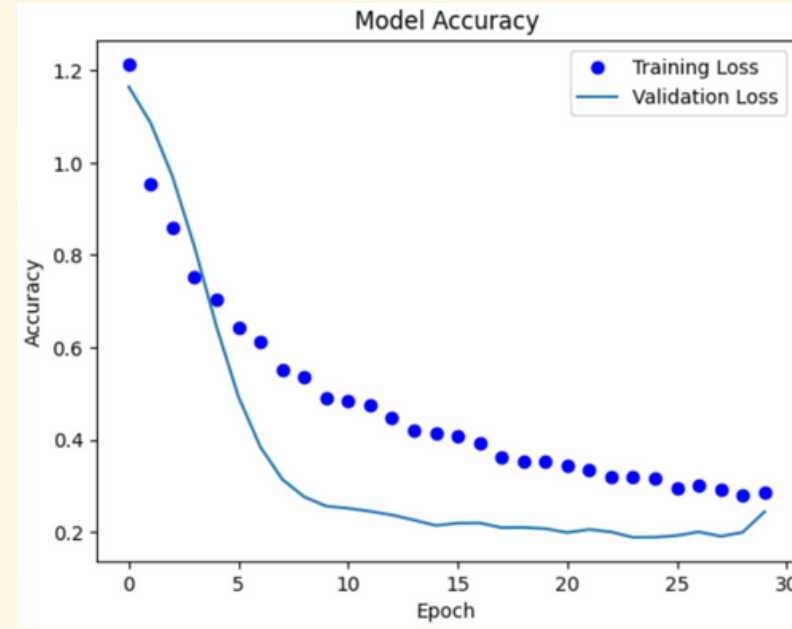
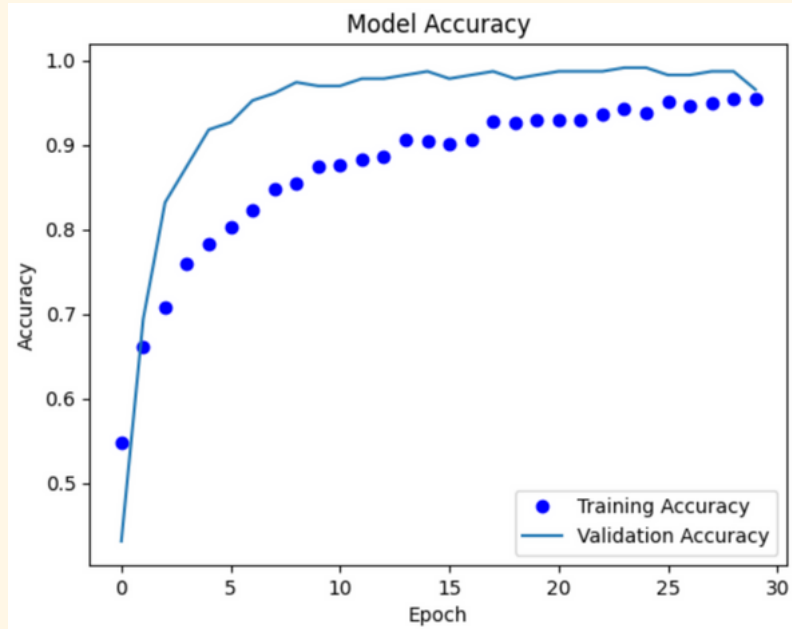
Model #7

- Adam Opt Learning rate: $1e-5$
- Conv2D: 1 Layers with activation "Relu" and max-pooling (32 filters)
- Dropout: 5 layers (0.5, 0.5, 0.3, 0.3, 0.3)
- Dense: 3 layers (512,256,128)
- Steps per Epoch: len(train_generator)
- Epoch:10
- Batch size 50

Model #11

- Adam Opt Learning rate: $1e-5$
- Conv2D: 3 Layers with activation "Relu" and max-pooling (32,64,128 filters)
- Dropout: 5 layers (0.5, 0.5, 0.5, 0.5, 0.3)
- Dense: 3 layers (512,256,128)
- Steps per Epoch: len(train_generator)
- Epoch:10
- Batch size 50

PREVIOUS MODELS



Model #18

- Adam Opt Learning rate: $1e-5$
- Conv2D: 1 Layers with activation "Relu" and max-pooling (32 filters)
- Dropout: 5 layers (0.5, 0.5, 0.3, 0.3, 0.3)
- Dense: 3 layers (512,256,128)
- Steps per Epoch: `len(train_generator)`
- Epoch: 30
- Batch size: 50

Model #25

- Adam Opt Learning rate: $1e-5$
- Conv2D: 4 Layers with activation "Relu" and max-pooling (32,64,128,256 filters)
- Dropout: 3 layers (0.3, 0.3, 0.3)
- Dense: 2 layers (64,32)
- Steps per Epoch: `len(train_generator)`
- Epoch: 15
- Batch size: 16

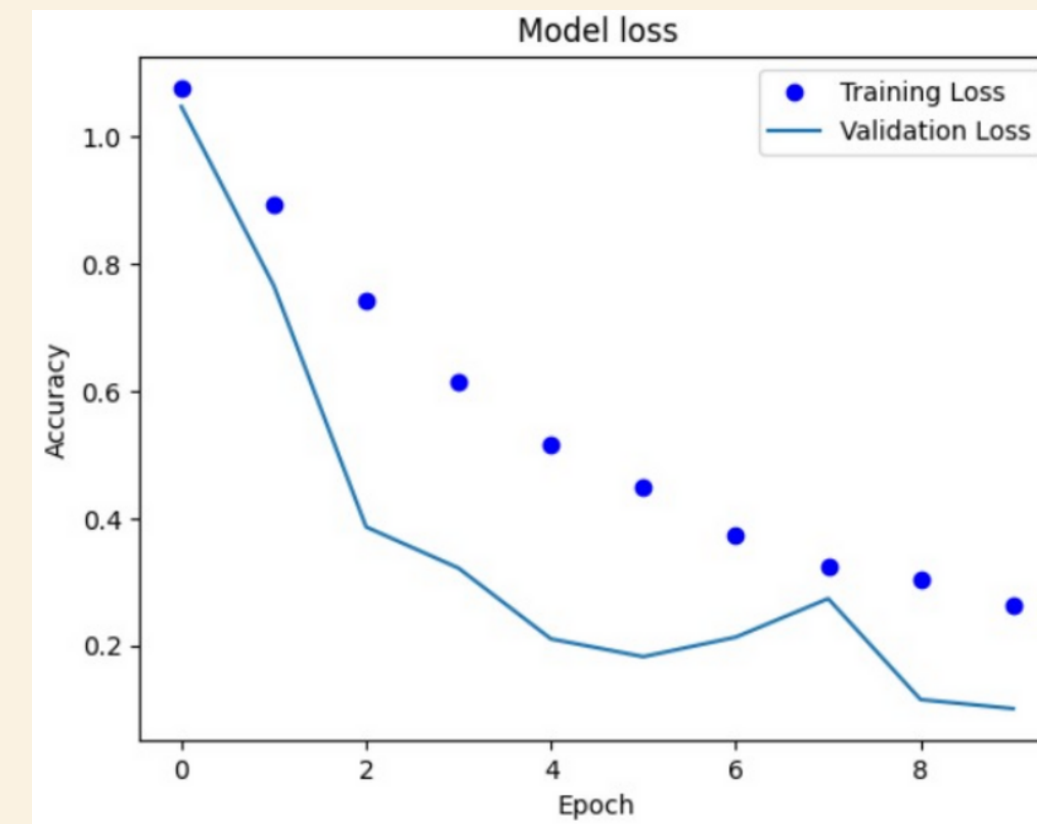
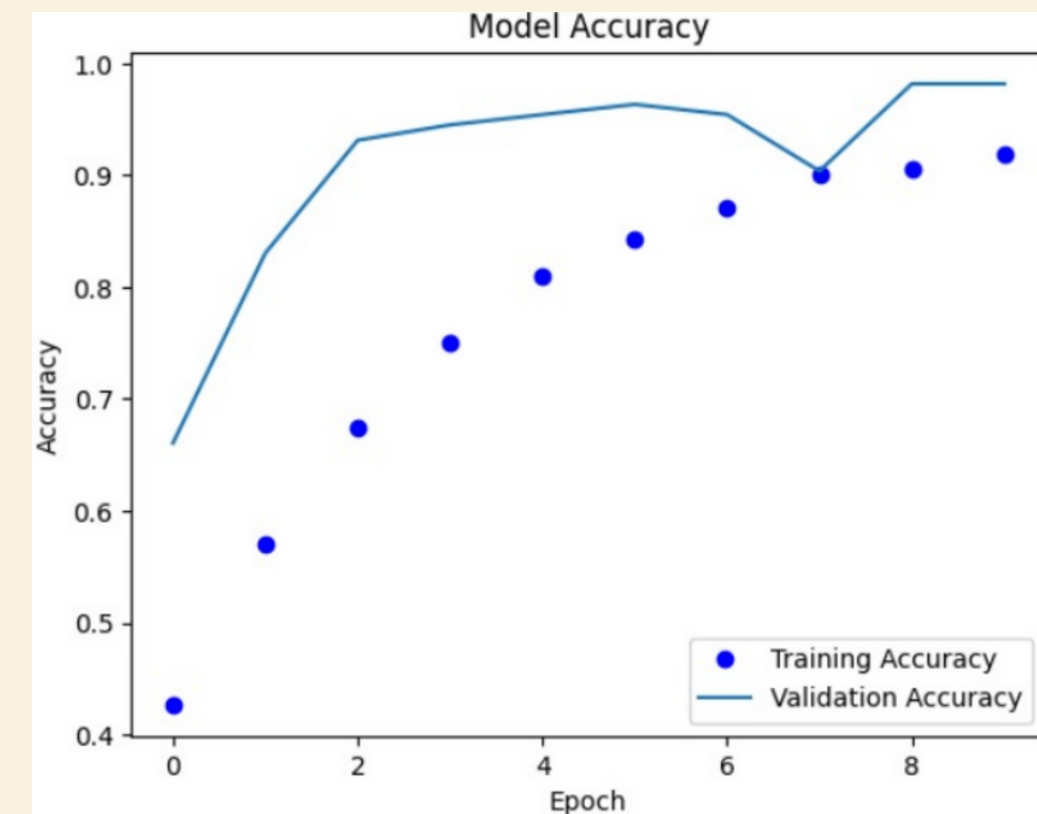
```
Test Data Evaluation
14/14 - 1s - loss: 0.1158 - accuracy: 0.9771 - 815ms/epoch - 58ms/step
Test Loss, Test Accuracy: [0.11579684168100357, 0.9770641922950745]
```

BEST TRAINED MODEL

- Adam Opt Learning rate: $1e-5$
- Conv2D: 3 Layers with activation "Relu" and max-pooling (32,64,128,256 filters)
- Dropout: 3 layers (0.6, 0.4, 0.2)
- Dense: 2 layers (64,32)
- Steps per Epoch: len(train_generator)
- Epoch:10
- Batch size: 16

Accuracy and Loss at Final Epoch

- Training Accuracy: 0.9398
- Validation Accuracy: 0.9519
- Training Loss: 0.1937
- Validation Loss: 0.1642



```
Epoch 10: saving model to C:/Python/2120795_StanleyLin_Pineapple_2112422_WaiYanAung_Strawberry\osu_model_400_e10.hdf5
245/245 [=====] - 104s 426ms/step - loss: 0.1937 - accuracy: 0.9398 - val_loss: 0.1642 - val_accuracy: 0.9519
```

MODEL SUMMARY

- 3 Convolutional Layers were used with each having Max Pooling.
- Batch Normalisation improves accuracy.
- The layers were flattened to convert the multidimensional input into a single linear vector for the dense layer.

Model: "model"

Layer (type)	Output Shape	Param #
=====		
conv2d_input (InputLayer)	[(None, 224, 224, 3)]	0
conv2d (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_2 (Conv2D)	(None, 56, 56, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 28, 28, 128)	0
conv2d_3 (Conv2D)	(None, 28, 28, 256)	295168
max_pooling2d_3 (MaxPooling2D)	(None, 14, 14, 256)	0
batch_normalization (BatchNormalization)	(None, 14, 14, 256)	1024
flatten (Flatten)	(None, 50176)	0
dropout (Dropout)	(None, 50176)	0
dense (Dense)	(None, 64)	3211328
dropout_1 (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 32)	2080
dropout_2 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 3)	99
=====		
Total params: 3,602,947		
Trainable params: 3,602,435		
Non-trainable params: 512		

EVALUATION - TEST DATASET

- Evaluate Test Data using the model to produce Test Loss and Test Accuracy.
- Test Loss and Validation Loss is similar with a [0.0335] difference.
- Identical value between validation and test datasets

```
#Load Model
model2 = load_model(
    "C:/Python/2120795_StanleyLin_Orange_2112422_WaiYanAung_Apple/oau_model_400_e10.hdf5")
# Evaluate the test data using the Loaded model
print("Test Data Evaluation")
results = model2.evaluate(test_generator, batch_size=None, verbose=2)
print("Test Loss, Test Accuracy: ", results)
```

```
Test Data Evaluation
15/15 - 2s - loss: 0.1307 - accuracy: 0.9738 - 2s/epoch - 115ms/step
Test Loss, Test Accuracy: [0.130710631608963, 0.9737991690635681]
```

- Test Accuracy: 0.9738
- Validation Accuracy: 0.9519
- Test Loss: 0.1307
- Validation Loss: 0.1642

STREAMLIT WEB APP TEST RESULT (APPLE)

- Test Result of Apple using images from the Internet:
Probability
Apple: 0.9477
Orange: 0.0447
Unknown:0.0075

Orange-Apple-Unknown Prediction

This is a simple image classification web app to predict Orange-Apple-Unknown

Please upload an image file



Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

Browse files



apples-fruit-cut-leaf-wallpaper-preview.jpg 73.7KB



It is an Apple!

Probability (0: Apple, 1: Orange, 2: Unknown)

0	1	2
0.9477	0.0447	0.0075

STREAMLIT WEB APP TEST RESULT (ORANGE)

- Test Result of Orange using images from the Internet:

Probability

Apple: 0

Orange: 0.9998

Unknown:0.0002

Orange-Apple-Unknown Prediction

This is a simple image classification web app to predict Orange-Apple-Unknown

Please upload an image file



Drag and drop file here

Limit 200MB per file • JPG, PNG, JPEG

Browse files



oranges.jpg 74.6KB



It is an Orange!

Probability (0: Apple, 1: Orange, 2: Unknown)

0	1	2
0	0.9998	0.0002

STREAMLIT WEB APP TEST RESULT (UNKNOWN)

- Test Result of Unknown using random object images from the Internet:
Probability
Apple: 0.1455
Orange: 0.0179
Unknown:0.8366

Orange-Apple-Unknown Prediction

This is a simple image classification web app to predict Orange-Apple-Unknown

Please upload an image file

Drag and drop file here
Limit 200MB per file • JPG, PNG, JPEG

Browse files

pen.jpg 33.8KB



It is unknown!

Probability (0: Apple, 1: Orange, 2: Unknown)

0	1	2
0.1455	0.0179	0.8366



ORANGE LIVE DETECT VIDEO





APPLE LIVE DETECT VIDEO

It is an Unknown! Accuracy: 97.54%

CONCLUSION AND FINDING

We faced several problems in data preparation:

- The first set of photos we took did not have a large variety of background colors, leading to an inability to detect the fruits under a messy background. To overcome this, we made sure we minimally had RGB, black and white background moving forward.
- The second set of photos was good, but we made a huge mistake of sharpening it to 100x100 pixels, with that small of a picture, even as the person who took that picture, it was hard to tell what's the image about. The image sharpening did not help much either. We then suspect that our model is not training properly due to this. Hence, we switched to 400x400 pixel images





CONCLUSION AND FINDING

How we maintained a high quality dataset

- We made sure to resize images to 400x400 to maintain a lower size with good image quality, furthermore images are sharpened for a clear quality
- We kept track of our model for comparison in ranking our best model and testing with live detect tools and streamlit tool.

Wow factors

- Using image sharpening with the Pillow library functions
- Image Resizing using PowerToys
- Shows real-time accuracy during live detect
- live detect using droidcam

CONCLUSION AND FINDING

How we countered overfitting

- Regularisation was introduced to help prevent overfitting
- We kept the number of each class Orange, Apple, and Unknown images to 1.4k $\pm$ 100. This prevents overemphasis on a particular class
- Good dataset with various backgrounds, angles, and lighting
- Dropout layer was introduced to help prevent overfitting

Quality of the model is determined by:

- Dataset quality
- Regularisation and Optimisation
- Architecture of the model



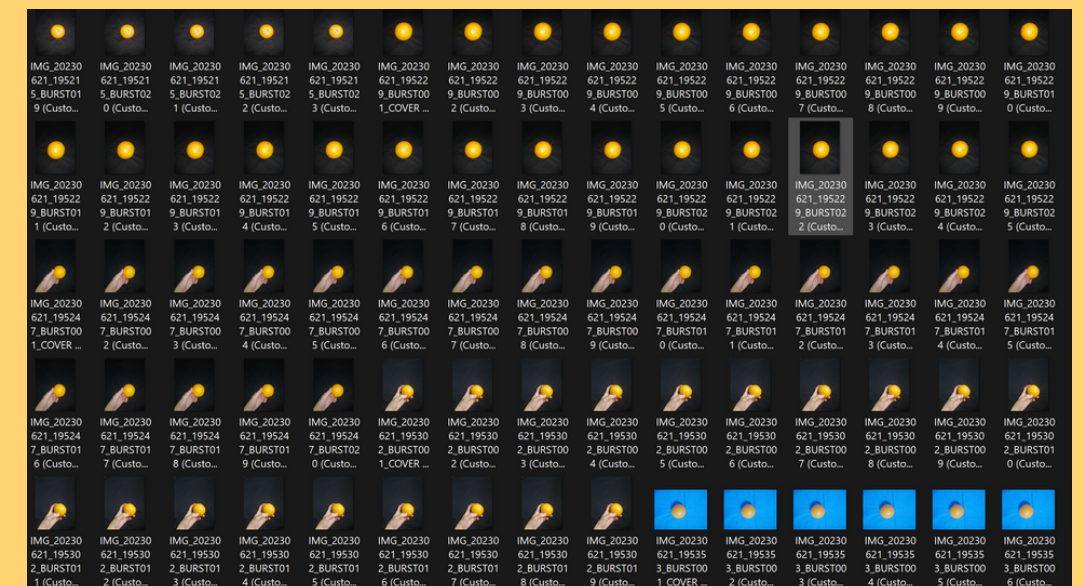
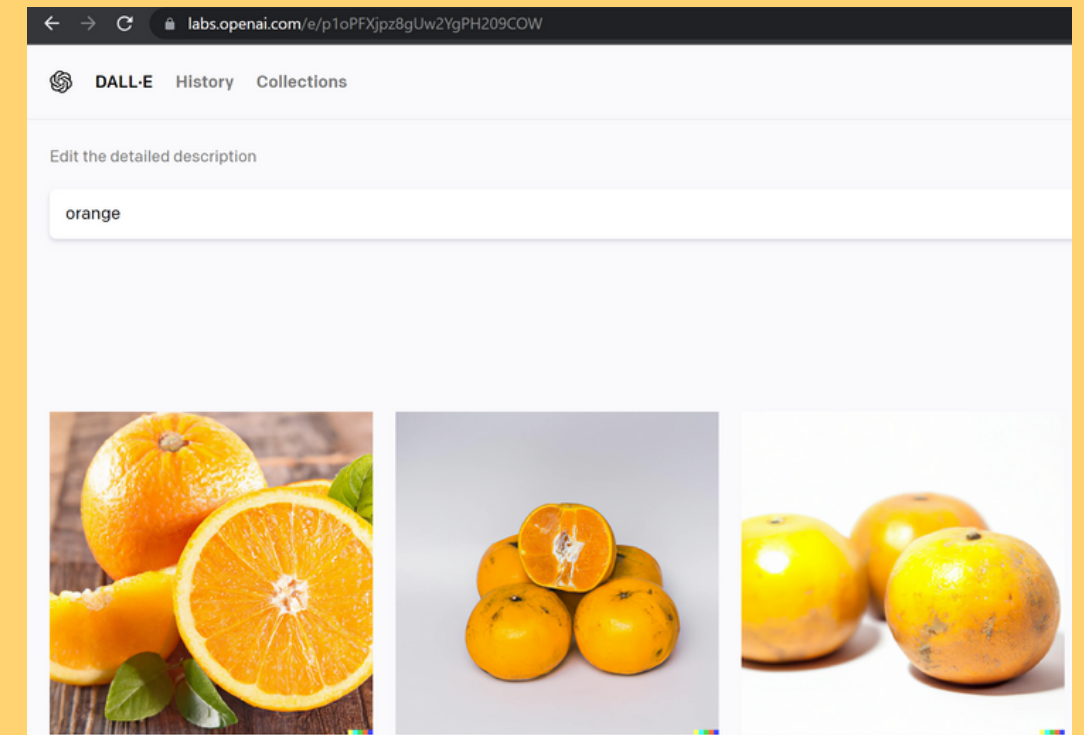
IMPROVEMENTS

Expansion in dataset

We could have expand our fruit dataset with additional varieties of the same fruit.

Decrease in burst shot

We could have taken fewer burst shots with the same angle, background, and lighting. This could increase the integrity of validation feedback when the model is under training.





**THANK YOU
FOR WATCHING**

